

INGENIERÍA EN COMPUTACIÓN II:

SISTEMA DE MONITOREO DE CALIDAD DEL AIRE
BASADO EN IoT

Agostina Braccacini – Julieta Córdoba – Luisina Milanese

Universidad Nacional de Rafaela (UNRaf)

Ingeniería en Computación

Materia: Ingeniería en Computación 2

Año: 2026

Resumen

El monitoreo de la calidad del aire en espacios cerrados ha cobrado creciente relevancia por su impacto directo sobre la salud y el bienestar de las personas, junto con la disponibilidad cada vez mayor de sensores de bajo costo que permiten medirla de forma continua. El presente trabajo describe el rediseño y la evolución de un sistema de monitoreo de calidad del aire basado en IoT, desarrollado originalmente en la materia Ingeniería 1 mediante un microcontrolador Arduino Uno y una aplicación móvil construida en MIT App Inventor.

El sistema fue migrado hacia una arquitectura más robusta, compuesta por un microcontrolador ESP32, un sensor de gases MQ-135, un broker de mensajería MQTT, un backend desarrollado en Django que persiste los datos en una base de datos de series temporales InfluxDB, y una interfaz web que consume una API REST propia. El trabajo presenta el marco teórico de las tecnologías involucradas, las motivaciones detrás de cada cambio arquitectónico, y la arquitectura final del sistema junto con su interfaz resultante.

Como ampliación de las funcionalidades originales, se desarrolló un sistema de control de ventilación cuya lógica fue validada mediante un módulo de relé conectado al ESP32, activando el indicador luminoso incorporado del propio módulo para simular el accionamiento de un sistema de ventilación. El sistema combina un funcionamiento automático, que activa el indicador cuando la concentración de gases alcanza un nivel considerado peligroso, con un control manual accesible desde la interfaz web mediante un selector de modo. De esta manera, el usuario puede alternar entre ambos modos de funcionamiento e intervenir sobre el estado del sistema al pasar a modo manual, evitando conflictos entre la decisión automática y la orden del usuario.

Palabras clave: Internet de las Cosas, monitoreo ambiental, ESP32, MQTT, Django, InfluxDB, calidad del aire.

Abstract

Air quality monitoring in enclosed spaces has become increasingly relevant given its direct impact on health and well-being, together with the growing availability of low-cost sensors that enable continuous measurement of this environmental variable. This paper presents the redesign and evolution of an IoT-based air quality monitoring system, originally developed in the Ingeniería 1 course using an Arduino Uno microcontroller and a MIT App Inventor mobile application.

The system was migrated toward a more robust architecture composed of an ESP32 microcontroller, an MQ-135 gas sensor, an MQTT message broker, a Django backend that persists data into an InfluxDB time-series database, and a web interface consuming a purpose-built REST API.

To expand upon the original functionalities, a ventilation control system was developed; its logic was validated using a relay module connected to the ESP32, activating the module's built-in indicator light to simulate the actuation of a ventilation system. The system combines automatic operation—activating the indicator when gas concentrations reach a level deemed dangerous—with manual control accessible via the web interface through a mode selector. This allows the user to switch between both operating modes and intervene in the system's status once manual mode is engaged, preventing conflicts between the automatic decision and the user's command.

Keywords: Internet of Things, environmental monitoring, ESP32, MQTT, Django, InfluxDB, air quality.

1. Introducción

La calidad del aire en espacios cerrados es una variable ambiental de creciente interés, tanto por su impacto directo sobre la salud y el bienestar de las personas como por la disponibilidad cada vez mayor de sensores de bajo costo que permiten monitorearla de forma continua. En los últimos años, el paradigma de Internet de las Cosas (IoT) ha facilitado la implementación de sistemas de monitoreo distribuido, en los que dispositivos embebidos equipados con sensores capturan variables físicas del entorno

y las transmiten a través de redes de comunicación hacia servidores que procesan, almacenan y exponen la información para su consulta (ITU-T, 2012).

Este trabajo se enmarca en un proyecto iniciado en la materia Ingeniería 1, en la cual se desarrolló una primera versión de un sistema de monitoreo de calidad del aire compuesto por un sensor de gases MQ-135 conectado a una placa Arduino Uno, con una aplicación móvil construida en MIT App Inventor como interfaz de visualización. Si bien esta primera implementación cumplió su objetivo pedagógico inicial, presentaba limitaciones significativas en materia de conectividad inalámbrica, capacidad de procesamiento y escalabilidad de la capa de presentación.

Con el propósito de superar estas limitaciones, en esta nueva etapa se rediseñó la arquitectura del sistema incorporando un microcontrolador ESP32 con conectividad WiFi, un broker MQTT para la comunicación entre dispositivos, un backend desarrollado en Django encargado del procesamiento de la información, una base de datos InfluxDB para el almacenamiento de series temporales y una interfaz web que consume una API REST para visualizar las mediciones en tiempo real e históricas.

Como complemento a las funcionalidades de monitoreo, se incorporó un sistema de control de ventilación cuya lógica fue validada mediante un LED conectado al ESP32, utilizado para simular el funcionamiento de un sistema de ventilación. Este mecanismo permite combinar el accionamiento automático cuando la calidad del aire alcanza niveles considerados peligrosos con el control manual desde la interfaz web, demostrando la capacidad del sistema no solo para supervisar el ambiente, sino también para actuar sobre él.

De esta manera, el proyecto evoluciona desde un prototipo de adquisición de datos hacia una plataforma IoT más completa, modular y escalable, capaz de integrar monitoreo, almacenamiento, visualización y mecanismos de actuación dentro de una misma arquitectura tecnológica.

1.1 Objetivo general

El objetivo general del proyecto consistió en desarrollar un sistema integral de monitoreo de calidad del aire que permitiera capturar datos de un sensor de gases en tiempo real, transmitirlos de forma inalámbrica hacia una infraestructura de procesamiento centralizada, almacenarlos de manera persistente en una base de datos especializada, y visualizarlos a través de una interfaz web accesible desde cualquier dispositivo conectado a la red del establecimiento.

1.2 Objetivos específicos

Para alcanzar este objetivo general se definieron una serie de objetivos específicos. En primer lugar, resultó necesario reemplazar la arquitectura de hardware inicial, basada en una placa Arduino Uno, por un microcontrolador con conectividad WiFi integrada, capaz de comunicarse directamente con la red sin depender de módulos externos adicionales. De manera complementaria, se buscó sustituir la aplicación móvil desarrollada en MIT App Inventor por una solución de visualización más robusta y escalable, que permitiera una mayor integración con servicios externos y un control más completo sobre la capa de presentación del sistema.

A partir de estas decisiones de base, se planteó diseñar e implementar un servicio backend propio que desacoplara el procesamiento de los datos tanto del broker de mensajería como de la base de datos, otorgando un mayor control sobre el formato de los mensajes y sobre el destino final de la información. Sobre esa misma infraestructura, se propuso construir una API REST que expusiera los datos de forma estandarizada, facilitando su consumo tanto por el frontend propio del sistema como por eventuales aplicaciones externas.

Asimismo, se estableció como objetivo diseñar una carcasa protectora para el sensor y la placa mediante herramientas de modelado 3D, con el propósito de garantizar la integridad física de los componentes electrónicos y facilitar una futura instalación en el ambiente monitoreado. Finalmente, se planteó incorporar un mecanismo de control de ventilación operable desde la interfaz web, de modo que el sistema pudiera evolucionar

desde un esquema de monitoreo pasivo hacia una solución capaz de intervenir activamente sobre las condiciones ambientales medidas.

2. Marco teórico

El desarrollo de un sistema de monitoreo ambiental basado en IoT requiere la aplicación de conceptos y tecnologías propias de la Ingeniería de Software y de los sistemas embebidos. En este proyecto se adoptaron herramientas, protocolos y arquitecturas ampliamente utilizados en el desarrollo de aplicaciones IoT, los cuales constituyen el fundamento teórico sobre el que se diseñó el sistema.

2.1 Internet de las Cosas y monitoreo ambiental

De acuerdo con la Recomendación Y.2060 de la Unión Internacional de Telecomunicaciones (ITU-T, 2012), el Internet de las Cosas puede definirse como una infraestructura global para la sociedad de la información que permite ofrecer servicios avanzados mediante la interconexión de objetos físicos y virtuales, apoyada en tecnologías de información y comunicación interoperables, existentes y en evolución. Esta definición pone el énfasis en dos aspectos centrales para el presente proyecto: la interconexión de objetos físicos —en este caso, un sensor de gases— con sistemas de información, y la interoperabilidad entre tecnologías, que se traduce en el uso de protocolos y formatos estándar para que los distintos componentes del sistema puedan comunicarse entre sí sin depender de un proveedor único.

En el dominio específico del monitoreo ambiental, la aplicación de IoT ha permitido superar varias limitaciones de los métodos tradicionales de medición, que suelen depender de estaciones fijas costosas y de baja densidad geográfica. La incorporación de sensores de bajo costo conectados a microcontroladores con capacidad de comunicación inalámbrica posibilita desplegar redes de monitoreo más densas y económicas, capaces de transmitir datos en tiempo real hacia plataformas centralizadas de almacenamiento y visualización.

2.2 Sensores de gases: el MQ-135

El sensor MQ-135 es un dispositivo de bajo costo ampliamente utilizado en proyectos de monitoreo de calidad del aire. Según su hoja de datos técnica, el sensor emplea dióxido de estaño (SnO_2) como material sensible al gas, cuya resistencia eléctrica disminuye en presencia de concentraciones crecientes de determinados gases contaminantes (Hanwei Electronics, s. f.). El sensor está compuesto por un tubo cerámico de micro- AL_2O_3 , la capa sensible de SnO_2 , electrodos de medición y un elemento calefactor, alojados dentro de una carcasa de plástico y malla de acero inoxidable (Hanwei Electronics, s. f.).

El MQ-135 es sensible a una amplia variedad de gases, entre ellos amoníaco, óxidos de nitrógeno, alcohol, benceno, humo y dióxido de carbono, lo que lo hace apropiado para aplicaciones de detección general de calidad del aire más que para la medición precisa de un único contaminante. Su salida puede ser analógica, proporcional a la concentración de gas detectada, y su fabricante recomienda un período de precalentamiento y una calibración previa para establecer el valor de resistencia en aire limpio que sirve de referencia para las mediciones posteriores.

En este proyecto, el sensor MQ-135 se utilizó para obtener una estimación del nivel de contaminación del aire mediante la lectura de su salida analógica a través del conversor analógico-digital (ADC) del ESP32. A partir de dichas lecturas, el sistema clasifica la calidad del aire en tres rangos discretos definidos en el firmware y en el simulador: «ambiente limpio» (para lecturas menores o iguales a 700), «ambiente regular» (entre 701 y 1500) y «ambiente peligroso» (valores superiores a 1500 dentro del rango de 12 bits del conversor ADC, de 0 a 4095). Esta categorización determina el estado informado en la carga útil de telemetría y rige el encendido o apagado automático del actuador de ventilación.

2.3 Microcontroladores para IoT: el ESP32

El ESP32 es una familia de microcontroladores de bajo costo y bajo consumo energético desarrollada por Espressif Systems, que integra conectividad WiFi y Bluetooth (Espressif Systems, s. f.). A diferencia de placas como el Arduino Uno, que carecen de

conectividad inalámbrica nativa, el ESP32 incorpora de fábrica los componentes necesarios para la transmisión de datos inalámbrica, entre ellos conmutadores de antena, amplificadores de radiofrecuencia y módulos de gestión de energía (Espressif Systems, s. f.), lo que simplifica considerablemente el diseño de dispositivos IoT al concentrar en un mismo componente tanto la adquisición de datos como su transmisión hacia la red.

Estas características hicieron del ESP32 una alternativa adecuada para reemplazar la plataforma Arduino utilizada en la primera versión del proyecto, permitiendo integrar en un único dispositivo la adquisición de datos, el procesamiento local y la comunicación inalámbrica.

2.4 El protocolo MQTT

MQTT (Message Queuing Telemetry Transport) es un protocolo de mensajería liviano basado en el patrón de publicación/suscripción, estandarizado por OASIS y reconocido como estándar ISO/IEC (OASIS, 2019). Fue diseñado específicamente para entornos con recursos limitados, lo que lo convierte en el protocolo de referencia para comunicaciones máquina a máquina e IoT (OASIS, 2019). En este modelo, los dispositivos productores de datos publican mensajes hacia un broker asociados a un tópico, mientras que los dispositivos consumidores se suscriben a los tópicos de su interés, desacoplando completamente a productores de consumidores de información. En este proyecto se estableció un contrato formal de tres tópicos bajo QoS 0 para la interacción entre componentes:

aq/aula-1/mq135: Tópico sobre el cual el ESP32 (o el simulador) publica periódicamente cada 4 segundos la telemetría del sensor en formato JSON con la estructura {"g": <valor>, "estado": "<clasificación>"}

aq/aula-1/vent/set: Tópico de control donde el backend publica comandos en texto plano ("ON", "OFF", "AUTO" o "MANUAL") para forzar la conmutación remota o alternar el modo de funcionamiento. Los comandos "ON"/"OFF" fuerzan el estado del relé y activan

el modo manual; "AUTO" devuelve el control al umbral automático del sensor; "MANUAL" cambia a modo manual sin alterar el estado actual del relé.

aq/aula-1/vent/estado: Tópico en el cual el ESP32 (o el simulador) responde en formato JSON con la estructura {"fan_on": <bool>, "manual": <bool>}, notificando el estado físico confirmado del actuador y el modo de funcionamiento vigente.

2.5 Bases de datos de series temporales: InfluxDB

InfluxDB es una base de datos especializada en el almacenamiento y la consulta de series temporales, optimizada para cargas de trabajo con altas tasas de escritura y consultas que agregan o filtran datos por rangos temporales, lo que la hace especialmente apropiada para aplicaciones de monitoreo IoT y análisis en tiempo real (InfluxData, s. f.). Cada punto de datos se organiza mediante un nombre de medición, un conjunto de etiquetas, un conjunto de campos con los valores medidos y una marca temporal, estructura que permite calcular eficientemente promedios, máximos, mínimos y evoluciones históricas.

Su utilización permite conservar el historial completo de mediciones y realizar consultas temporales que posteriormente son utilizadas para la construcción de gráficos y estadísticas en la interfaz web.

2.6 Arquitectura REST y el framework Django

REST (Representational State Transfer) es un estilo arquitectónico para el diseño de sistemas distribuidos, propuesto por Fielding (2000) como una abstracción de los principios de diseño subyacentes a la arquitectura de la World Wide Web. Entre sus características principales se destacan la separación entre cliente y servidor, la ausencia de estado en las interacciones y el uso de una interfaz uniforme basada en recursos direccionables mediante URIs, lo que lo hace particularmente adecuado para sistemas IoT que exponen datos simples como la última lectura de un sensor.

Django es un framework de desarrollo web de alto nivel escrito en Python (Django Software Foundation, s. f.). Sobre esta base se construyeron tanto el servicio de ingestión y almacenamiento de datos de telemetría como la API REST que expone esos

datos hacia el frontend, implementada mediante vistas de Django que devuelven respuestas en formato JSON.

2.7 Arquitectura Cliente-Servidor

El sistema desarrollado responde al modelo de arquitectura cliente-servidor. En este esquema, el backend implementado en Django actúa como servidor, encargado del procesamiento de datos, la lógica de negocio y el acceso a la base de datos, mientras que la interfaz web funciona como cliente consumiendo la API REST para presentar la información al usuario. Esta separación favorece la modularidad, la mantenibilidad y la escalabilidad de la aplicación .

3. Metodología

El desarrollo del sistema se llevó a cabo mediante una metodología iterativa e incremental, tomando como punto de partida el prototipo construido durante la asignatura Ingeniería 1. En lugar de desarrollar una solución completamente nueva, se optó por analizar la arquitectura existente e introducir mejoras progresivas sobre cada uno de sus componentes, validando su funcionamiento antes de continuar con la siguiente etapa.

Este enfoque permitió reducir riesgos durante el desarrollo, reutilizar el conocimiento adquirido en la versión inicial y verificar de forma continua el funcionamiento del sistema a medida que se incorporaban nuevas tecnologías y funcionalidades.

3.1 Análisis de la arquitectura existente

La primera etapa consistió en analizar el funcionamiento del sistema desarrollado en Ingeniería 1, identificando las limitaciones de la placa Arduino Uno en materia de conectividad y procesamiento, así como las restricciones de MIT App Inventor en cuanto a integración con servicios externos y escalabilidad de la interfaz.

3.2 Migración del hardware

A partir del análisis anterior se decidió reemplazar el Arduino Uno por un microcontrolador ESP32, capaz de resolver la conectividad inalámbrica y la comunicación con el broker MQTT sin componentes adicionales.

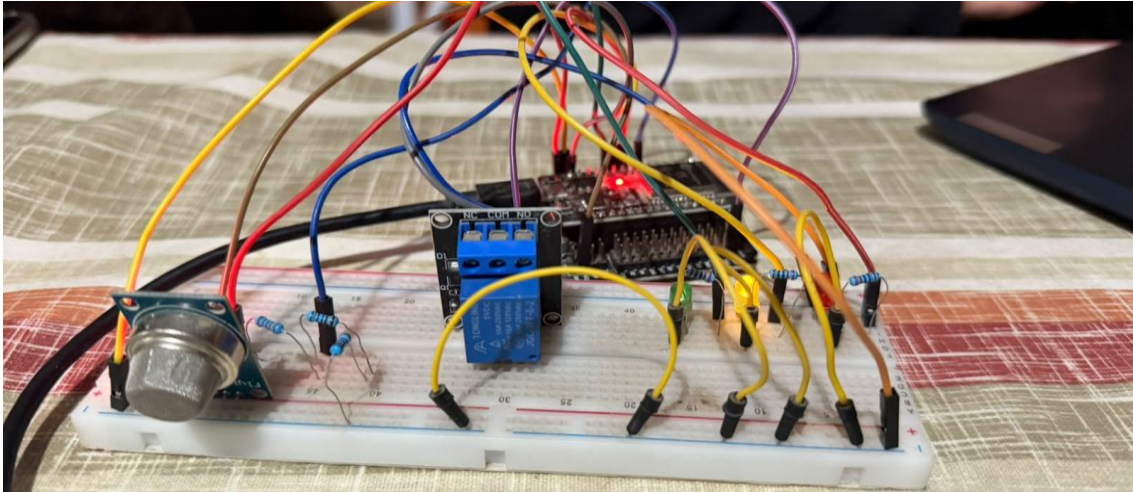


Ilustración 1 - Circuito armado en placa de prototipado, con el sensor MQ-135, el microcontrolador ESP32, el módulo de relé y los indicadores LED de calidad del aire.

3.3 Diseño del servicio de telemetría

Se evaluó inicialmente el uso de Node-RED para vincular el broker MQTT con la base de datos, optando finalmente por desarrollar un servicio propio en Django que otorgara mayor control sobre el procesamiento y el destino de los mensajes.

3.4 Diseño de la API REST y del frontend

Con los datos ya persistidos en InfluxDB, se diseñó una API REST que expone los servicios necesarios para el frontend, y se desarrolló una interfaz web independiente que consume dichos servicios mediante solicitudes HTTP.

3.5 Diseño físico

Como etapa final del proyecto se diseñó una carcasa mediante herramientas de modelado 3D con el objetivo de integrar, organizar y proteger los componentes electrónicos del sistema.

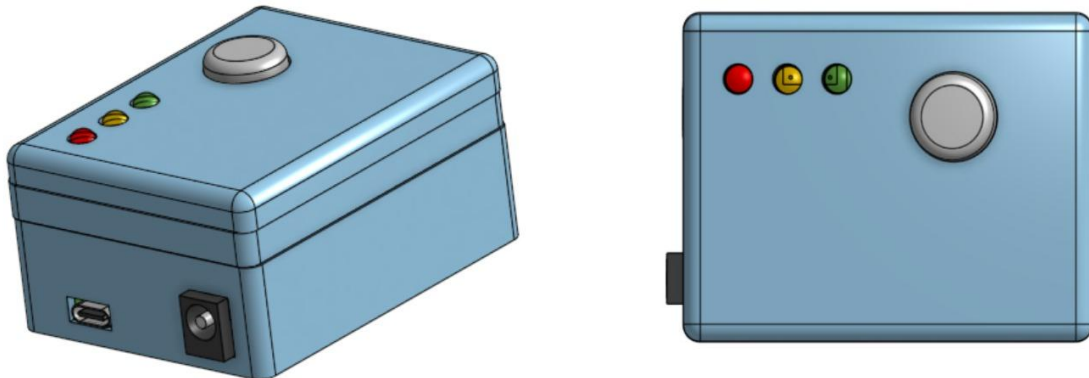


Ilustración 2 - Diseño de la carcasa protectora modelada en Onshape: (a) vista isométrica; (b) vista frontal con los indicadores LED y el sensor.

3.6 Implementación del simulador de hardware

Para garantizar la testabilidad del sistema en entornos donde no se cuenta con el dispositivo físico conectado, se desarrolló un simulador en Python (`simulator.py`) basado en la librería `paho-mqtt`. El simulador implementa de manera idéntica el contrato de tópicos del microcontrolador: publica periódicamente lecturas analógicas aleatorias controladas en `aq/aula-1/mq135`, escucha peticiones en `aq/aula-1/vent/set` y emite confirmaciones en `aq/aula-1/vent/estado`. Adicionalmente, cuenta con un hilo de ejecución por consola que permite inyectar valores manuales (de 0 a 4095), facilitando la verificación de extremo a extremo de las capas de transporte, backend y presentación ante diferentes umbrales de contaminación.

3.7 Uso de herramientas de inteligencia artificial

Durante el desarrollo de este trabajo se utilizó Claude (Anthropic) como herramienta de asistencia para la depuración de código, la generación de diagramas explicativos y la revisión de redacción, bajo supervisión y validación de las autoras en cada caso.

4. Desarrollo del proyecto

4.1 Descripción general del proyecto

El sistema desarrollado tiene como propósito la medición, transmisión y visualización en tiempo real de la calidad del aire en un ambiente de uso académico. Su arquitectura está compuesta por un microcontrolador ESP32, un sensor MQ-135, un broker MQTT, un backend en Django que persiste los datos en InfluxDB, y una interfaz web que consume una API REST propia.

Como resultado de este proceso de evolución se obtuvo una plataforma IoT compuesta por dispositivos embebidos, servicios backend y una interfaz web integrada, capaz de adquirir, transmitir, almacenar, visualizar y gestionar información sobre la calidad del aire de manera centralizada.

4.2 Arquitectura general del sistema

La arquitectura final del sistema se diseñó siguiendo un enfoque en capas, con el objetivo de desacoplar las distintas responsabilidades de la aplicación y facilitar su mantenimiento, escalabilidad y futura evolución. A diferencia de la primera versión desarrollada en Ingeniería 1, donde la adquisición, el procesamiento y la visualización de la información se encontraban estrechamente integrados, la nueva propuesta distribuye estas funciones en componentes independientes que se comunican mediante protocolos estandarizados.

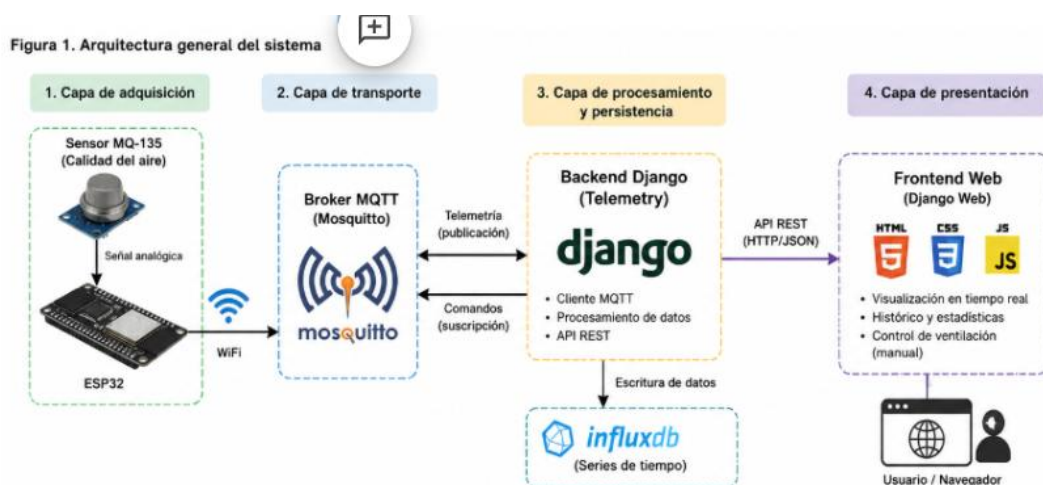


Ilustración 3 - Arquitectura general del sistema.

Como se observa en la Figura 3, la arquitectura se organiza en cuatro capas principales:

Capa de adquisición: integrada por el sensor de gases MQ-135 y el microcontrolador ESP32. El sensor realiza las mediciones de la calidad del aire y el ESP32 procesa dichas lecturas para publicarlas en el broker MQTT mediante conexión WiFi.

Capa de transporte: implementada mediante un broker MQTT (Mosquitto), responsable de gestionar el intercambio de mensajes entre el dispositivo IoT y el backend. Además de transportar las mediciones enviadas por el ESP32, esta capa permite recibir comandos provenientes de la aplicación web para el control del sistema de ventilación simulado.

Capa de procesamiento y persistencia: desarrollada utilizando Django. Este servicio se suscribe a los tópicos MQTT, procesa las mediciones recibidas, las almacena en una base de datos InfluxDB y expone una API REST que permite acceder a la información desde aplicaciones externas.

Capa de presentación: conformada por una aplicación web desarrollada con Django, HTML, CSS y JavaScript, encargada de consumir la API REST para mostrar las mediciones en tiempo real, consultar el historial de datos y permitir la interacción del usuario con el sistema.

Esta organización en capas desacopladas permitió distribuir las responsabilidades del sistema, facilitando tanto su mantenimiento como la incorporación de futuras funcionalidades.

4.3 Evolución del hardware: de Arduino Uno a ESP32

En la primera instancia del proyecto la conexión con el sensor MQ-135 se realizaba mediante una placa Arduino Uno, la cual carece de conectividad WiFi integrada y presenta una capacidad de procesamiento reducida. Estas limitaciones obligaban a resolver la conectividad de red mediante módulos externos adicionales. Por este motivo se reemplazó el Arduino Uno por un ESP32, lo que permitió disponer de conexión

inalámbrica integrada en el mismo componente encargado de la lectura del sensor, así como publicar los datos directamente en el broker MQTT sin intermediarios.

Además del incremento en la capacidad de procesamiento, la incorporación del ESP32 permitió simplificar el hardware del sistema al eliminar la necesidad de módulos externos de comunicación inalámbrica.

4.4 Evolución del software de usuario

El proyecto original contemplaba MIT App Inventor para el desarrollo de la aplicación móvil de visualización, herramienta con limitaciones significativas en la integración con servicios externos y en la escalabilidad de la interfaz. Con el avance del proyecto, esta herramienta fue reemplazada por una solución basada en tecnologías web estándar (HTML, CSS y JavaScript), servida a través de una aplicación Django independiente, lo que garantizó mayor control sobre la capa de presentación y una integración directa con la API REST desarrollada.

4.5 Reemplazo de Node-RED por un servicio propio en Django

En una etapa intermedia, la comunicación entre el broker MQTT y la base de datos se resolvía mediante Node-RED. Sin embargo, se identificó la necesidad de contar con mayor control sobre el formato de los mensajes y el destino de los datos, por lo que se optó por desarrollar un servicio propio en Django. Para ello se configuró un entorno virtual en una Raspberry Pi y se creó un proyecto Django denominado *airsrv*, dentro del cual se implementó la aplicación *telemetry*. Esta aplicación incluye un cliente MQTT que se suscribe al tópico `aq/aula-1/mq135`, deserializa la carga útil en JSON y persiste el registro en una base de datos InfluxDB v1 denominada *calidadaire*. Cada punto temporal almacena la marca de tiempo, el valor entero medido en el campo `gas` (proveniente de la clave `g`) y el estado cualitativo (ambiente limpio, regular o peligroso) para su posterior explotación mediante la API REST.

Esta arquitectura permitió centralizar la lógica de negocio del sistema, facilitando el mantenimiento y la incorporación de nuevas funcionalidades.

4.6 Diseño de la API REST

Sobre los datos almacenados en InfluxDB se construyó una API REST que expone los siguientes servicios: */api/latest* (valor actual del sensor y estado del sistema), */api/query* (histórico de las últimas N mediciones), */api/statistics* (promedio, máximo, mínimo y conteo de lecturas) y */api/health* (verificación rápida del estado del backend). Este diseño desacopla explícitamente el frontend de la infraestructura interna del sistema, de modo que cualquier cambio futuro en el broker o en la base de datos no debería impactar en los clientes que consumen la API.

4.7 Frontend y visualización de datos

El frontend se implementa como un segundo proyecto Django, independiente del servicio de telemetría, que sirve una interfaz compuesta por HTML, CSS y JavaScript. El archivo *dashboard.js* utiliza la función *fetch()* para consumir periódicamente los endpoints de la API REST. Los datos obtenidos se organizan en tarjetas informativas — valor actual, estado del ESP32 y estadísticas— y en gráficos que representan tanto el histórico completo de mediciones como el promedio agrupado por hora.

La actualización periódica de la información permite al usuario supervisar continuamente el estado del ambiente sin necesidad de recargar manualmente la página.

4.8 Flujo de funcionamiento del sistema

Una vez definida la arquitectura general, resulta de interés analizar el flujo de información entre los distintos componentes que conforman el sistema. La Figura 2 presenta tanto el recorrido de los datos de telemetría desde el sensor hasta la interfaz web como el flujo de control utilizado para accionar el sistema de ventilación simulado desde la aplicación.

Figura 2. Flujo de funcionamiento del sistema

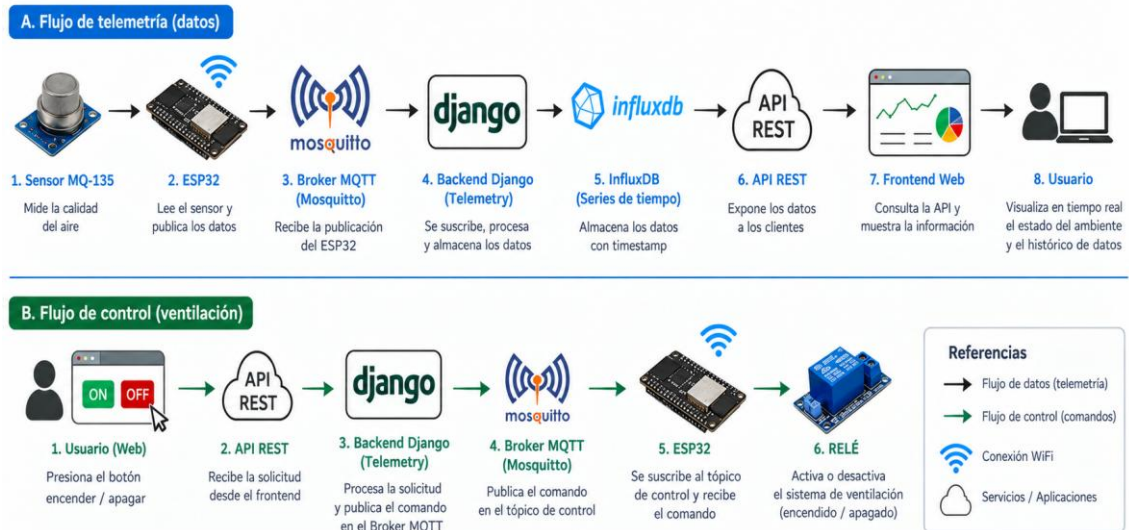


Ilustración 4 - Flujo de funcionamiento del sistema: telemetría (A) y control de ventilación (B).

En el flujo de telemetría, el sensor MQ-135 obtiene la medición de calidad del aire y el ESP32 publica periódicamente los datos en el broker MQTT. El backend desarrollado en Django recibe dichas publicaciones, procesa la información y la almacena en InfluxDB, desde donde posteriormente es expuesta mediante una API REST para ser visualizada en la interfaz web.

De forma complementaria, el flujo de control permite que el usuario interactúe con el sistema desde el navegador. Al accionar el botón de control, el frontend envía la solicitud al backend, el cual publica un mensaje MQTT hacia el ESP32. Finalmente, el dispositivo recibe el comando y actualiza el estado del relé que simula el funcionamiento del sistema de ventilación.

4.9 Diseño y montaje físico

El armado físico incluyó el ensamblaje del circuito electrónico —sensor MQ-135 conectado al ESP32 mediante una placa de prototipado— y el diseño de una carcasa protectora modelada en Onshape, herramienta de diseño asistido por computadora en la nube. El diseño contempla indicadores LED visibles, una abertura circular que permite que la parte sensible del sensor permanezca en contacto con el aire del ambiente y

aberturas para los conectores de alimentación y datos, priorizando tanto la protección de los componentes como la usabilidad del dispositivo en una futura instalación.

4.10 Interfaz de visualización

Como resultado del desarrollo se obtuvo una interfaz web accesible que permite observar en tiempo real el funcionamiento del sensor y consultar el histórico de mediciones. Al acceder a la página principal se presenta el título "Monitor de calidad del aire — Aula 1" junto con la hora de la última actualización.

La interfaz organiza la información en cuatro tarjetas principales, actualizadas automáticamente sin necesidad de recargar la página. La primera muestra el nivel actual del sensor expresado como valor analógico (ADC), el estado cualitativo del ambiente (limpio, regular o peligroso) y la fecha y hora de la última lectura recibida. La segunda tarjeta presenta las estadísticas de las últimas 24 horas: promedio, máximo y mínimo del valor medido. La tercera tarjeta informa el estado del dispositivo ESP32, indicando si se encuentra activo, el tiempo transcurrido desde la última señal recibida. La cuarta tarjeta corresponde al control de ventilación, mostrando el estado actual del relé (encendido o apagado), un indicador textual del modo de funcionamiento vigente (automático o manual), un botón para alternar entre ambos modos y un botón para encender o apagar el relé, habilitado únicamente cuando el sistema se encuentra en modo manual.

Debajo de las tarjetas se despliega el histórico de mediciones mediante un gráfico de líneas que representa la evolución temporal del valor del sensor, junto con un gráfico de barras que muestra el promedio de las lecturas agrupado por hora.

La siguiente imagen muestra la interfaz web desarrollada.

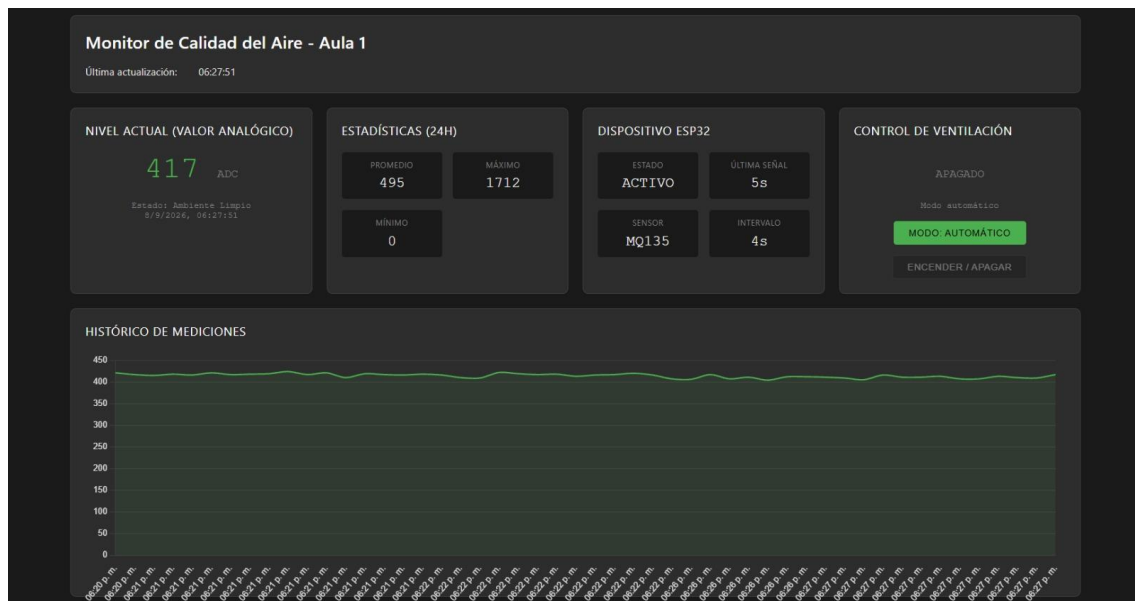


Ilustración 5 - Interfaz web del sistema de monitoreo de calidad del aire.

4.11 Extensión del sistema: control de ventilación

Como ampliación del sistema original, se incorporó un mecanismo de control de ventilación operado desde la misma plataforma web utilizada para la visualización de datos.

El objetivo de esta extensión es que el sistema, además de registrar y visualizar los datos de calidad del aire, pueda actuar sobre las condiciones medidas tanto de forma manual como automática.

En la modalidad manual, la interfaz web incorpora un botón que permite al usuario alternar entre el modo automático y el modo manual del sistema. Mientras el sistema permanece en modo automático, el botón de encendido y apagado del relé se muestra deshabilitado, evitando que se emitan órdenes manuales que entrarían en conflicto con la decisión automática vigente. Una vez activado el modo manual, dicho botón se habilita y permite al usuario encender o apagar un módulo de relé conectado al ESP32, cuyo indicador luminoso incorporado permite verificar visualmente su accionamiento, independientemente de los valores detectados por el sensor. Al presionar el botón de encendido o apagado, el frontend envía la orden correspondiente hacia el backend, que

la publica como mensaje de comando en el broker MQTT bajo un tópico diferenciado del utilizado para la telemetría; el ESP32, suscripto a dicho tópico, recibe la orden y actualiza el estado del relé.

En la modalidad automática, el ESP32 evalúa continuamente las mediciones obtenidas por el sensor MQ-135 y, cuando el valor registrado supera el umbral definido como peligroso, activa automáticamente el relé. De esta manera, el dispositivo responde de forma autónoma ante condiciones ambientales desfavorables.

Cabe destacar que, en la implementación actual, el módulo de relé se utiliza como mecanismo de validación de la lógica de control, accionando su propio indicador luminoso a modo de prueba, sin encontrarse aún conectado a una carga externa como un ventilador físico. La incorporación de dicha carga queda planteada como una de las líneas de trabajo futuro del proyecto.

5. Resultados

El desarrollo del proyecto permitió obtener un sistema IoT funcional para el monitoreo de la calidad del aire, capaz de adquirir mediciones mediante un sensor MQ-135, transmitir las utilizando el protocolo MQTT, procesarlas mediante un servicio desarrollado en Django y almacenarlas de forma persistente en una base de datos InfluxDB. Sobre esta infraestructura se implementó una API REST que expone la información al frontend web, permitiendo la consulta de datos en tiempo real e históricos desde cualquier dispositivo conectado a la red.

Como complemento al monitoreo ambiental, se implementó un mecanismo de actuación cuya lógica fue validada mediante un módulo de relé conectado al ESP32, utilizado para simular el funcionamiento de un sistema de ventilación. El sistema permite tanto la activación automática del indicador cuando se detectan niveles elevados de contaminación como su control manual desde la interfaz web mediante el intercambio de mensajes MQTT.

6. Conclusiones

El presente trabajo permitió diseñar e implementar una plataforma IoT para el monitoreo de la calidad del aire, integrando dispositivos embebidos, protocolos de comunicación, servicios backend, bases de datos y una interfaz web.

La incorporación del microcontrolador ESP32, el protocolo MQTT, el framework Django, la base de datos InfluxDB y una API REST hizo posible la construcción de una solución más flexible, escalable y mantenible, preparada para integrar nuevos dispositivos y funcionalidades sin modificar la estructura general del sistema.

Como parte de esta evolución, el sistema dejó de limitarse al monitoreo de variables ambientales para incorporar un mecanismo básico de actuación, validado mediante un módulo de relé conectado al ESP32 cuyo indicador luminoso incorporado permite simular el funcionamiento de un sistema de ventilación. Esta funcionalidad demostró la integración entre la adquisición de datos, el procesamiento de la información y la ejecución de acciones sobre el entorno a partir de las condiciones detectadas por el sensor.

Finalmente, la arquitectura desarrollada constituye una base sólida para futuras extensiones del proyecto. Entre las posibles líneas de trabajo se encuentran la incorporación de un ventilador físico conectado a la carga del relé, en reemplazo del indicador luminoso utilizado durante la etapa de validación y el despliegue de la plataforma en otros entornos donde resulte necesario realizar el monitoreo de la calidad del aire en tiempo real.

Bibliografía

- Django Software Foundation. (s. f.). *Django documentation*. <https://docs.djangoproject.com/>
- Espressif Systems. (s. f.). *ESP32 Series Datasheet*. <https://www.espressif.com/en/support/documents/technical-documents>
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-Based Software Architectures* (Doctoral dissertation, University of California, Irvine). <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Hanwei Electronics Co., Ltd. (s. f.). *MQ-135 Gas Sensor Technical Data*. <https://www.alldatasheet.com/datasheet-pdf/pdf/1132551/HANWEI/MQ-135.html>
- InfluxData. (s. f.). *InfluxDB documentation*. <https://docs.influxdata.com/>
- ITU-T. (2012). *Recommendation ITU-T Y.2060: Overview of the Internet of Things*. International Telecommunication Union. <https://www.itu.int/rec/T-REC-Y.2060>
- Mosquitto Project. (s. f.). *Eclipse Mosquitto documentation*. <https://mosquitto.org/documentation/>
- OASIS. (2019). *MQTT Version 5.0*. OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- Onshape. (s. f.). *Onshape Learning Center*. <https://learn.onshape.com/>